AgentPulse

Speaker notes for the project presentation. Thirteen slides, budgeted at roughly 18 minutes of speaking with 7 minutes left for questions. Every number in these notes appears somewhere in the repository and can be shown on request.

How to run the talk

The deck is built so the strongest material lands in the middle, not at the end. Slides 1 to 7 establish that the system works. Slide 8 is the turning point and is the reason this project is worth presenting: three self-authored benchmarks were checked against external data and all three broke. Slides 9 to 13 turn that into a method, a limitation list and an open question.

Resist the temptation to soften slide 8. A panel that hears you volunteer three failures will trust the successes on the other slides far more than a panel that hears only successes.

Per-slide notes

Slide 1 - Title

~40 seconds

Open with: "AgentPulse evaluates every span a multi-agent system produces, on CPU, inside your own infrastructure."

- Say 'never sampled' explicitly. It is the one-line differentiator and everything else follows from it.
- Flag the shape of the talk now: the most interesting part is not what worked, but what failed external validation.
- Do not linger. The title slide earns nothing; the panel wants the problem.

If you say only one sentence here: every span, on CPU, self-hosted.

Slide 2 - The problem

~1 min 30 s

Open with: "A multi-agent system can fail in ways that leave no error and no exception behind."

- Walk the four rows briefly. Do not read them verbatim, the panel can read.
- Land the closing line: these faults are rare by nature, so sampling is structurally the wrong instrument for them.
- That single sentence is the justification for the entire architecture on the next slide.

The point: rare faults plus sampled evaluation equals faults you never see.

Slide 3 - Architecture

~1 min 30 s

Open with: "The reason full coverage is affordable is that evaluation was taken off the request path."

- Trace the five stages left to right once, quickly.
- The ingest API loads no inference models at all. That is what brought the API process from 1.24 GB to 0.10 GB.
- Durability is not theoretical: the process was killed mid-evaluation and the job was recovered and run exactly once.
- If evaluation were inline you would be forced back to sampling, which defeats the whole thesis.

The architectural claim: decoupling is what makes 100% coverage possible at all.

Slide 4 - The evaluator cascade

~1 min 45 s

Open with: "A cheap gate in front of an expensive judge, and the ablation that justifies it."

- Stage 1 is MiniLM cosine similarity at 19.1 ms. Stage 2 is a DeBERTa-v3 cross-encoder, only on the uncertain cases.
- End-to-end the two models measure 151.19 ms per case (PyTorch path, ablation run of 2026-09-18; the 2026-08-23 run of the same script gave 215.9 ms). Be precise about this: 19.1 ms is MiniLM alone, not the pipeline.
- On the chart: the cascade matches DeBERTa alone at F1 0.963 while being cheaper on the common case. That is the justification.
- Point at configuration F deliberately. Adding the old drift signal made the classifier worse, 0.619. That result was published rather than dropped, and the next slide but one explains why it was wrong.

Thresholds were selected on the dev split and applied unchanged to the held-out test split.

Slide 5 - Four signals and their maturity

~1 min 30 s

Open with: "Four signals ship, and each one carries an honest maturity label inside the product."

- Grounding and drift are Beta and validated. Disagreement and tool-claim are Experimental and are not.
- Say clearly that the label appears on screen in the product, not only in the report.
- Most tools in this space present every capability as equally ready. Stating the tier is a deliberate credibility choice.

This slide is the thesis of the project in miniature.

Slide 6 - Drift: the detector that was rebuilt

~2 min

Open with: "The synthetic benchmark said drift detection worked. Real agent text said it fired on 91.7% of normal operation."

- Tell it as a sequence: synthetic looked fine, real text destroyed it, diagnosis showed the threshold was never the problem.
- The measured centroid distance never exceeded 0.099 against a 0.30 threshold, so the signal had in fact never fired.
- The fix was the representation: a baseline window mean against a current window mean, pooled and disjoint.
- Then state the cost yourself: coverage is 24.5%. When this detector speaks it is accurate; it stays silent on about three quarters of sessions.

Calibrated on 89 dev tasks with the criterion fixed beforehand, then measured once on 111 held-out tasks.

Slide 7 - Engineering results

~1 min 15 s

Open with: "Four production properties, each measured rather than argued for."

- Exactly-once recovery across 8,000 spans: zero lost, zero retried, zero duplicated.
- Roughly 12 spans per second at four workers. Eight workers buys 8% more throughput for 86% more memory, so four is the operating point.
- ONNX is 1.97 times faster than PyTorch with a worst-case probability difference of 1.2e-08, which is to say identical.
- Keep this slide brisk. It is credibility, not the argument.

Every number here came from running the thing, not from reasoning about the design.

Slide 8 - The turning point

~2 min 30 s - the most important slide

Open with: "Three signals were checked against external data that this project did not author. All three broke."

- Go row by row and do not rush. Drift: 91.7% false alarms. Tool-claim: zero claims extracted from 8,353 real prose spans. Disagreement: zero of ten contradictions detected.
- Contrast each against its internal score. Tool-claim scored 0.842 on its own 19 cases. Disagreement scored 0.960 on 22 authored cases.
- Then read the bottom line and pause: three for three is not bad luck, it is a finding about how self-authored benchmarks get built.
- Let the silence sit for a beat before moving on.

If the panel remembers one slide, it should be this one.

Slide 9 - Why they failed

~2 min

Open with: "In both cases the benchmark measured the wrong shape of problem."

- Tool-claim: the extractor looks for an agent narrating 'I used the X tool'. Modern harnesses put the tool name in a structured field and never narrate it. Expanding the regex cannot fix this, because the information is not in the text.
- Disagreement: the 22 internal cases averaged ten words, minimal pairs of exactly the shape the NLI model was trained on. Real debate turns run past 2,000 characters. The benchmark handed the detector pre-extracted claims, so the missing extraction stage was invisible.
- Spend the most time on the evidence-partition finding at the bottom. Two agents can flatly contradict each other and both be correct about their own partition of the evidence.
- Six of forty labelled cases were exactly that. An NLI score compares two strings and has no representation of what each agent could see.

This is the strongest technical point available. It reframes a failed feature as a well-posed research question.

Slide 10 - Method

~1 min 30 s

Open with: "These findings were only possible because of four rules that were fixed in advance."

- Benchmark before fixing. The disagreement engine was measured against completely unmodified code first.
- Report the contradiction. The relevance gate measured worse than baseline in isolation, 0.762 against 0.800. Only the combination paid off, and the inconvenient intermediate result was published.
- Separate the label from the judge. Scoring an LLM judge against LLM-produced labels is circular, so results were split by label provenance.
- Install the competitor rather than reading its marketing. Two positioning claims were refuted by running Phoenix and MLflow.

A result can be attacked. A method that deliberately arranged to be proven wrong is much harder to attack.

Slide 11 - Where it stands

~1 min

Open with: "What is actually working today, and what was deliberately left out."

- 209 of 209 backend tests pass. Over 20,700 spans stored. Five versioned evaluation datasets.
- Run through the shipped list quickly; the detail is on the slide.
- End on the deferred list and say the reasons are stated, not hidden: multi-tenancy, key rotation, backup and restore, Postgres, OTLP.

If asked why 'production ready' never appears: it is binary and unfalsifiable. The claim made instead is specific and checkable.

Slide 12 - Limitations

~1 min 15 s

Open with: "Five limits, stated before the panel has to find them."

- Deliver these calmly and without apology.
- Drift coverage is 24.5%. Two of four signals are unvalidated. The API key is single and shared. Evaluation coverage is currently about 6% because most stored spans predate the worker.
- Datadog was never audited because it cannot be installed, so its column in the competitive matrix is marked as the least reliable.

Naming your own limits converts a likely attack into evidence of rigour.

Slide 13 - Close

~1 min

Open with: "The contribution is a working system and an honest evaluation of it."

- Four parts: a working system, one validated capability, three reported negative results, and one open research question.
- Close on the fourth, not the first. Anyone can demo a dashboard.
- Final line: this project turned its instruments on itself, published what it found, and ended with a sharper question than it started with.

Then stop talking and take questions.

Questions the panel is likely to ask

Three of your four signals failed. Is the project a failure?

No, and the distinction matters. One signal, drift, was diagnosed and rebuilt and now measures AUC 0.991 on a held-out split. The other two produced a documented reason for failure rather than a mystery. A project that reports three negative results with diagnoses has more evidence behind it than one that reports four successes with none checked.

Why should we trust the drift number when coverage is only 24.5%?

You should trust it exactly as far as it is stated. The claim is not that drift is always detected; it is that when this detector emits a value, false alarms sit at 1.5% and AUC at 0.991 on a held-out split. Coverage is quoted in the same breath every time, including inside the product.

Why not just use an LLM as a judge?

It was measured head to head. The local Qwen3-8B judge scored F1 1.000 against the cascade's 0.963, so on quality the judge won. But it cost 12.9 times the mean latency and 219 generation tokens per evaluation against zero. At 100% coverage that difference decides the architecture. Also, results were split by label provenance, and on the deterministically labelled subset both systems tie at 1.000.

What stops this from being a thin wrapper over an NLI model?

The evaluation is the smaller half. The durable leased queue with exactly-once recovery, the Alembic-managed schema, the retention path that deletes with zero orphans, the self-monitoring surface and the drift baseline that survives a restart are all separate engineering problems that had to be solved to make full coverage viable.

How is this different from LangSmith, Phoenix or Langfuse?

It is not competitive on breadth and the report says so. Phoenix and MLflow were installed and probed, and both refuted a claim this project had been making about tool-call verification. What survives is drift, which neither ships as a named feature or as composable primitives, plus the design choice of evaluating every span rather than a sample.

Why is the tool-claim redesign not finished?

It is blocked on labelling, not engineering. A labelling attempt reached Cohen's kappa of only 0.225 and produced zero gold examples for two of the four target classes. The disagreement was systematic rather than noisy: the same model with differently worded prompts returned UNVERIFIABLE 29 times in one pass and twice in the other. That says the question is not well posed on this data, which is not something prompt tuning fixes.

What is the evidence-partition problem in one sentence?

Two agents holding different subsets of the evidence can produce statements that look like a flat contradiction while both are correct, and a similarity or entailment score over two strings has no way to tell that apart from a genuine fault.

What would you do next?

Three things, in order. Persist enough context to distinguish partial-evidence disagreement from a real contradiction. Extract individual assertions so each can be labelled against the single tool result it refers to, which is the task shape that previously reached kappa 0.922. And process the existing backlog so evaluation coverage rises from about 6% toward the design intent.

Does it scale?

To roughly 100,000 traces on SQLite with WAL, yes, and that was the measured target. Beyond that the honest answer is that Postgres and horizontal partitioning are deferred with stated reasons rather than implemented. Scale was never the binding constraint; framework breadth and automatic issue surfacing are.

Numbers worth having ready

Grounding cascade F1	0.963	held-out v1.0_test, Config C
MiniLM only / DeBERTa only / shipped	19.1 / 132.1 / 151.2 ms	ablation_results.json, 2026-09-18
Drift false alarms	91.7% to 1.5%	before and after the rebuild
Drift detection / AUC	0.9192 / 0.991	111 held-out tasks
Drift coverage	24.5%	always quote with the accuracy

Durability	0 lost, 0 duplicated	8,000 spans, SIGKILL mid-run
Throughput	~12 spans/sec	4 workers, 8 physical cores
ONNX speedup	1.97x	worst prob difference 1.2e-08
API memory	1.24 GB to 0.10 GB	models moved off ingest
Tool-claim on real traces	F1 0.000	0 extractions, 8,353 prose spans
Disagreement, external	0 of 10	internal benchmark was 0.960
Backend tests	209 / 209	pytest tests/ -q

If a number is challenged, the underlying file is in experiments/results/ and the method is written up in the matching report at the repository root.